Original Article

Convo-Ease: A Policy-as-Prompt Gatekeeper for Real-Time Multimodal Moderation in Enterprise Communication

Soham S. Jadhav [1], Omkar N. Gadakh [1], Atharv Gaikwad [1], Nisha D. Patil [1]

1. *Artificial Intelligence and Data Science, MET's Institute of Engineering, Nashik, IND*

Abstract

Enterprise communication platforms carry text, image, and audio content across organizational contexts that public-facing moderation systems were never designed to serve. Existing moderation acts after content reaches recipients, depends on fixed harm taxonomies that cannot be updated without retraining, and rarely covers more than one or two modalities. Organizational settings need the opposite: validation before delivery, team-specific policies that change often, and coverage across every modality a message can arrive in. This paper contributes Convo-Ease, an applied moderation architecture defined by four design decisions and realized as a working system. A pre-delivery gatekeeper validates content structurally in the message-send path, so a message that fails evaluation is never shown to other members. A unified text abstraction reduces every modality to a single representation before evaluation, so one policy layer governs all three. A Policy-as-Prompt mechanism reads organizational rules as plain-language instructions at inference time, so a policy change is a configuration edit rather than a retraining job, and no model training is required at any point. A plugin-based backend separates each moderation task from the model that performs it, so a hosted API model and a local open-weights model are interchangeable per modality through configuration alone. We validate the central architectural claim on a held-out set of 300 human-labeled messages evaluated live under matched conditions on the same model with no parameter update between them: replacing a generic safety taxonomy with organization-specific natural-language rules raises recall from 46.0% to 76.7% and accuracy from 68.3% to 76.7%, a paired difference that is statistically significant (McNemar exact $p = 0.004$). The same mechanism holds across fifteen distinct organizational rule sets without any per-rule retraining, and an independent enforcement-intensity control shows a measured, asymmetric effect on the operating point. Live policy evaluation against a hosted model sustains sub-second batched latency, within the budget real-time chat requires. Convo-Ease demonstrates that an adaptable, multimodal, pre-delivery moderation system can be assembled from interchangeable off-the-shelf models plus an architecture, with no training in the loop, and that this design occupies a niche no existing system fills.

Categories: AI/ML-based decision support systems, Natural Language Processing (NLP)

Keywords: ai moderation, multimodal learning, organizational communication, text-image-audio fusion, rule-based moderation

Introduction

Digital messaging tools sit at the center of how organizations operate. Engineering teams coordinate on Slack or Microsoft Teams, sales teams run on WhatsApp, and customer-facing groups maintain dedicated channels for vendor and client contact. The same flexibility that makes these tools useful is where the operational risk begins. Conversations drift off-task, internal policies are broken, and sensitive information occasionally lands in a channel it should never have reached. None of these are exotic failures. They are everyday risks, and the moderation tooling available today does not match them, because most of it was built for public social platforms where the goal is to remove harmful content after a user reports it. In an internal channel, that workflow is already too late: once a confidential document or a policy-violating message is visible, the harm has begun, even if the message is deleted a minute later.

What organizations need is a different shape of moderation. It must screen content before it reaches recipients, apply team-specific policies in real time, adapt as those policies change, and work across whatever modality the content arrives in, whether text, image, or audio, rather than handling only one of them well.

This is a systems problem, not a modeling problem. The individual capabilities required, language understanding, image captioning, and speech transcription, already exist in off-the-shelf models. What does not exist is an architecture that arranges them so an organization gets pre-delivery validation, adaptable natural-language policy, and multimodal coverage at once, without retraining a model each time a rule changes, and without shipping content to an external classifier it does not control. Four limitations recur in current systems. Modality coverage is uneven, with text handled reliably, images partially, and audio rarely. Policy adaptability is poor, since updating a rule typically means retraining or re-engineering the model rather than editing a configuration [1,2]. Deployment context is ignored because methods built for community moderation or agent safety carry assumptions that do not transfer to enterprise chat [3]. And the validation point is wrong: messages are inspected after delivery, so a violating message has already been read by the time the system reacts.

The contribution of this paper is an architecture for real-time multimodal moderation, and a working system that realizes it. First, a pre-delivery gatekeeper architecture places moderation structurally in the message-send path as inline middleware, so a message that fails evaluation is persisted only in a withheld state and never enters the feed that other members read; the check-before-publish property is a property of the architecture, not of model accuracy. Second, a Policy-as-Prompt enforcement mechanism reads organizational rules authored in natural language at inference time, with a separate control that adjusts enforcement intensity independently of rule content, so a policy change takes effect on the next message with no retraining anywhere in the system. Third, a plugin-based backend architecture separates each moderation task from the model that performs it, so any modality's model, hosted API, or local on-premise, is substituted through one interface with no change to the moderation logic, which is what lets an organization run identical policy logic against an on-premise model for data-sovereignty reasons. Fourth, a unified text abstraction reduces image and audio to text before evaluation, so a single natural-language policy layer governs all three modalities without separate per-modality moderation logic.

The remainder of this article reviews the related work bearing on the design, presents the architecture and its Policy-as-Prompt and plugin mechanisms, describes the evaluation methodology, reports the architectural properties the system demonstrates and their measured support, and discusses implications, limitations, and positioning against existing systems.

Convo-Ease draws on and departs from several lines of work in automated moderation. The most relevant fall into three themes: text guardrails and policy adaptability, multimodal moderation, and the gap that motivates the present design.

Llama Guard is a safety classifier that scores content against a configurable taxonomy of harm categories, and its zero-shot configurability, expressing a new policy as a taxonomy update rather than a retraining run, is the property that informed our Policy-as-Prompt design [1]. Its limits are equally clear: text only, with no awareness of organizational policy as distinct from generic safety. LlamaFirewall layers prompt-injection detection, behavior auditing, and code inspection for agent safety [2]. The sharpest quantified gap is in policy adaptability: a leading static-taxonomy guardrail drops to 13.1% F1 when asked to enforce free-form, user-defined policies, a direct measure of how far fixed-taxonomy systems sit from what organizational moderation needs [4]. The closest conceptual neighbor to our approach encodes community moderation rules as natural-language instructions and reaches a median accuracy near 64% on Reddit communities, with most residual error attributable to prompt brittleness rather than reasoning [3]. Convo-Ease adopts that conceptual core, ports it to a multimodal enterprise setting, and later reproduces the underlying pattern, a fixed taxonomy underperforming a rule-aware alternative, as a controlled paired comparison within one system, holding the model fixed and varying only the policy content.

Adding images and audio changes the difficulty profile in ways that are not always intuitive. A multimodal benchmark across social-media tasks finds zero-shot performance consistently weak and the modality gap task-dependent rather than uniform [5]. A study contrasting video hate detection with image-text meme detection reports 0.848 macro-F1 on

the video task but only 0.628 AUROC on the meme task under one architecture, and recovers the video performance by using transcript-derived text before classification [6]. Image captioning has been shown to serve the same role, converting a visual moderation problem into a text problem [7], and a training-free defense converts unsafe image inputs to text so a model's existing safety alignment activates, yielding large safety gains with no fine-tuning [8]. These results are the empirical basis in the literature for Convo-Ease's decision to reduce every modality to text before evaluation.

Multimodal guardrails are advancing, but not yet far enough for the enterprise case. A recent extension of a text-only guardrail to image content handles only text and image, operates in English, and processes a single image at a time [9]. Adversarial benchmarks show that jailbreak strategies transfer from text-only to multimodal models with attack success above 50% on specific categories [10], and a multi-domain benchmark of nineteen guardrail models reports wide cross-domain variance and uniform adversarial fragility [11]. Across these lines of work, no existing system combines real-time coverage of text, image, and audio with adaptable natural-language policy enforcement and pre-delivery validation. That unoccupied combination is the niche Convo-Ease targets.

Materials And Methods

System Architecture

Convo-Ease is organized around five decisions that together produce the behavior pre-delivery enterprise moderation requires. Modular separation: each modality has its own processing path, and the three paths run independently behind a shared plugin interface, so an installation can adopt text moderation first and add image or audio later without architectural rework. Unified text abstraction: every input is converted to text before it reaches the policy evaluator, so text arrives as text, images are reduced to a short plain-text description, audio is transcribed and summarized, and a single policy layer governs all three. Externalized policy in two dimensions: rules are kept outside the model, authored in natural language by a group administrator, stored with the group record, and assembled into the evaluation prompt at inference time, and a separate sensitivity setting (Strict, Moderate, or Relaxed) controls how aggressively the same rules are enforced at the margin; rules and sensitivity are edited independently, and their effects are measured separately in the Results and Discussion. Structural gatekeeping: the check-before-publish property is enforced in the message-send path itself, so a message is evaluated before it is persisted in a deliverable state, and a flagged message is never returned in the visible feed. Built-in explainability: every decision carries a structured reason, so a blocked message records the rule-grounded explanation that triggered it, which makes the system auditable and gives an affected member a basis for appeal.

Convo-Ease moves a message through a sequence of stages, shown in Figure *1*. At input, the interface receives a text, image, or audio submission for a given group. In preprocessing and modality reduction, text passes through deterministic pre-filters (described below under the Policy-as-Prompt mechanism), an image is sent to a vision-language backend that returns a one or two-sentence plain-text description, and audio is transcribed and then summarized, so after this stage, every modality is represented as text. In policy integration, the unified text is assembled into an evaluation prompt together with the group's rules, the configured sensitivity, a context-use mode, and a bounded slice of recent conversation. In core moderation, the prompt is submitted to the language-model evaluator, which returns a verdict parsed into an allow or block decision with a reason. In output and enforcement, an allowed message is persisted and delivered, while a blocked message is persisted with a reason, withheld from the feed, and the sender is notified. Administrators author and edit rules, set sensitivity, and inspect a dashboard aggregating pass and flag rates, reasons, and per-member indicators from the decision log.

The architecture assigns each modality a role rather than a fixed model; this is summarized in Table *1*. The policy evaluator must reliably follow natural-language instructions over long contexts, since it reads the full policy, a conversation slice, and the message in a single pass. The image captioner must produce a faithful short description, and the transcriber must have an error rate and language coverage adequate for enterprise use without domain fine-tuning

[12,13]. Vision-language pre-training of the BLIP-2 family bridges image and text at real-time latency [14], and the fusion taxonomy of the multimodal representation literature maps onto these role assignments [15]. The plugin architecture described below formalizes the interface that makes these roles model-agnostic.

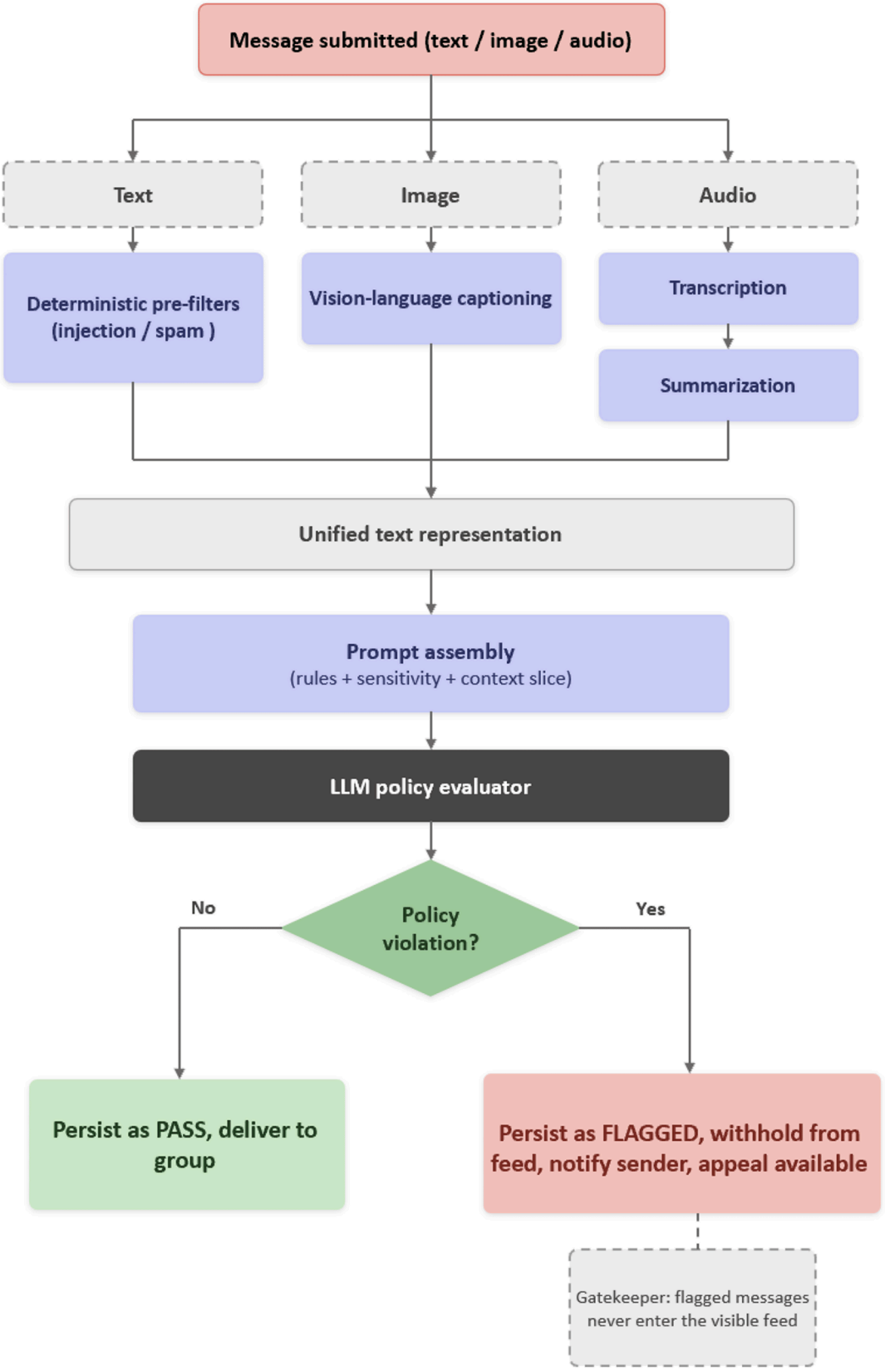

Message submitted (text / image / audio)
Text
Image
Audio
Deterministic pre-filters (injection / spam)
Vision-language captioning
Transcription
Summarization
Unified text representation
Prompt assembly
(rules + sensitivity + context slice)
LLM policy evaluator
Policy violation?
No
Yes
Persist as PASS, deliver to group
Persist as FLAGGED, withhold from feed, notify sender, appeal available
Gatekeeper: flagged messages never enter the visible feed

FIGURE 1: Convo-Ease moderation pipeline

A submitted message is reduced to text (images captioned, audio transcribed and summarized), evaluated against the assembled prompt by the language-model policy evaluator, and either delivered (PASS) or withheld (FLAGGED).
LLM: large language model.

Task	Prototype binding	Execution	On-premise alternative
Policy evaluation	Large open model via hosted API	Hosted	Local open-weights LLM
Image captioning	Vision-language model via hosted API	Hosted	Local BLIP-2 / LLaVA-class
Audio transcription	Whisper	Local	Same
Policy store	NL rules + sensitivity per group	Local	Same
Decision parsing	Structured PASS/FLAGGED, fail-safe	Local	Same
Pre-filters	Deterministic injection/spam/leetspeak	Local	Same
Application layer	REST send-path gatekeeper	Local	Same

TABLE 1: Convo-Ease task-to-model bindings in the prototype. The architecture is backend-agnostic; on-premise open-weights equivalents use the same interfaces.

The Policy-as-Prompt Mechanism

The Policy-as-Prompt mechanism turns an organizational rule into a moderation decision without retraining. It rests on the in-context instruction-following ability of modern language models [16] and is engineered to control the prompt brittleness that limits the natural-language policy approach in prior work [3].

A policy is a block of natural-language rules attached to a group, authored and edited by the administrator. Each group also carries a sensitivity setting, Strict, Moderate, or Relaxed, which is a separate control from the rules. The rules define the content boundary, what is or is not allowed. The sensitivity defines the enforcement intensity at that boundary, how aggressively borderline content is treated once a rule could plausibly apply. Because the policy lives with the group record rather than in model weights, changing a rule or the sensitivity takes effect on the next message with no retraining.

At inference time, the system assembles one evaluation prompt from the rules, the sensitivity instruction, a context-use mode, the recent-conversation slice, and the message in unified-text form. The prompt instructs the model to treat the rules as the primary decision boundary, to use context only for resolving short replies, pronouns, and references, and not

to flag a message merely for changing the subtopic. It includes explicit handling for emoji, evaluated by intent rather than appearance, and instructs the model to judge non-English content and character substitutions by meaning rather than literal characters. The production prompt template is shown below.

```
You are a strict Group Chat Moderator.

ADMIN RULES:
{rules}

MODERATION SENSITIVITY:
{moderation_sensitivity}
{sensitivity_instructions}

CHAT CONTEXT USE MODE:
{context_mode}

MODERATION PRINCIPLES:
1. The ADMIN RULES are the primary decision boundary.
2. CHAT CONTEXT is only for understanding references, short replies, follow-up questions, or pronouns.
3. Do not narrow a broad rule to the latest micro-topic in the chat.
4. If a message fits the ADMIN RULES on its own, it should PASS even if it changes the subtopic.
5. Only use context to rescue ambiguous messages like "what happened?", "yes", "same here", "when?".
6. Do not FLAG a message just because it differs from the last few messages, unless it clearly
 violates or falls outside the ADMIN RULES.

EMOJI HANDLING:
- Evaluate emoji by meaning and intent, not appearance.
- Evaluate skin-tone or combined emoji that could carry discriminatory intent by context.
- A single reaction emoji is almost always harmless.

CHAT CONTEXT:
{context_str}

TASK:
Validate the NEW message against the rules and context. Evaluate non-English content by meaning.
Be aware users may use character substitutions (leetspeak) to disguise prohibited content;
evaluate phonetic and visual meaning, not just literal characters.
Reply in one of these formats only:
PASS
FLAGGED <reason>

NEW MESSAGE: "{message}"{normalized_line}
```

The sensitivity instruction resolves to one of three texts: Strict flags borderline content and leans toward blocking when uncertain, Moderate (the default) enforces consistently without over-flagging, and Relaxed flags only clear and meaningful violations. The context-use mode is set to required for short or referential messages, and reference-only otherwise, so the evaluator leans on context only when the message cannot be judged alone.

The reply is parsed into a verdict. A reply beginning PASS is an allow; a reply beginning FLAGGED is a block whose trailing text becomes the stored reason. An empty or malformed reply is treated as a block with an explanatory reason rather than a silent pass. This is deliberate: in a pre-delivery gatekeeper, an ambiguous model output must not result in unmoderated delivery. The system also fails closed at the transport level, so a backend error or timeout blocks the message rather than delivering it unchecked.

Before any model call, a text message passes three deterministic filters. An injection filter matches known manipulation patterns (instructions to ignore prior directions, assume a new role, or force a PASS) and blocks with an injection-specific reason. A spam filter catches placeholder text and degenerate repetition. A leetspeak normalizer rewrites common substitutions and passes the decoded form to the model as a hint. These provide a fast, model-independent first pass whose measured standalone coverage is reported in the Results and Discussion; they are a supplement to, not a replacement for, the model layer.

Algorithm 1 formalizes the per-message decision for the text path. The image and audio paths are identical from the unified-text stage onward, differing only in the modality-reduction step that produces that text.

> Algorithm 1: Pre-delivery moderation decision (text path)
>
> Input: message m, group rules R, sensitivity s, recent context C
>
> Output: verdict v in {PASS, FLAGGED(reason)}
>
> 1: if R is empty then return PASS # no policy, no gate
>
> 2: if injection_pattern(m) then return FLAGGED(injection_reason)
>
> 3: if spam_pattern(m) then return FLAGGED(spam_reason)
>
> 4: m_dec := normalize_leetspeak(m) # decoded hint if changed
>
> 5: mode := context_mode(m) # required | reference_only
>
> 6: prompt := assemble(R, s, mode, C, m, m_dec)
>
> 7: out := LLM_evaluate(prompt) # constrained: PASS | FLAGGED <reason>
>
> 8: v := parse(out) # fail-safe on empty/ambiguous
>
> 9: return v

Plugin-Based Backend Architecture

A processing plugin defines a moderation task, moderates text, describes an image, transcribes audio, independent of any model that performs it. A backend factory binds each task to a concrete model at configuration time, either a hosted inference API or a local on-premise model, selected per modality. This separation is the architectural property, and it holds regardless of which model is plugged in at a given time. It directly answers a limitation shared by the text-only and multimodal systems reviewed above: a moderation architecture tied to one model's release cycle.

The same plugin interface accepts a hosted model behind an API or a local open-weights model, chosen per modality through configuration. A documented model-swap policy governs local execution: when local text and vision models share a constrained GPU memory budget, the engine releases one before loading the other, and on CPU-only hosts, it avoids swapping because disk reload would dominate latency. Changing which model serves a modality, or moving a modality from hosted to local execution, is a configuration change, not a rewrite of the moderation logic. This is the same edit-not-retrain property that Policy-as-Prompt gives for rules, extended to models.

Commercial moderation services require sending content to an external server for evaluation, which conflicts with data-sovereignty requirements in regulated industries, discussed further in the Results and Discussion. The plugin separation means an organization can run identical policy logic against an on-premise model with no architectural change. The prototype demonstrates both modes: it runs its text evaluator and image captioner through a hosted inference API and its transcriber locally, and the same interfaces accept fully local models for a sovereign deployment. The

interchangeability is exercised in this paper rather than asserted: the evaluation was produced by running the live-hosted configuration against a real model over real network round-trip times, with wall-clock latency captured per evaluation batch. Figure *2* shows the plugin and backend structure.

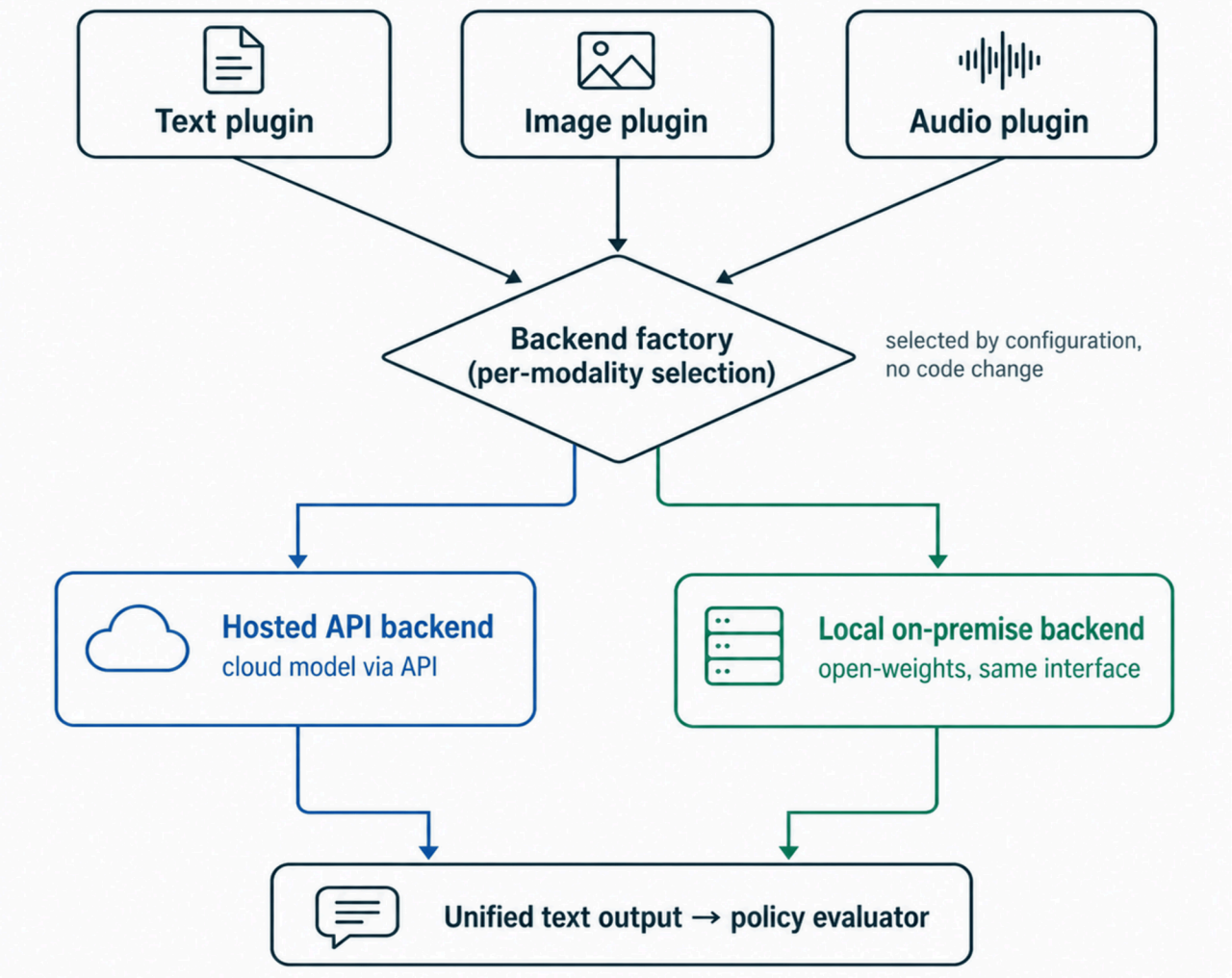


FIGURE 2: Backend-agnostic plugin architecture

Each modality plugin binds to either a hosted API backend or a local on-premise backend through the backend factory, selected by configuration with no change to the moderation logic.
API: application programming interface.

Evaluation Methodology

The evaluation is designed to test the architecture's claims, not to rank a model. It is organized so the central claim, that reading organizational policy as a prompt changes moderation outcomes, is isolated as a controlled variable, and so the system can be assessed without exposing real organizational chat.

The evaluation grounds itself in public, human-labeled moderation corpora mapped into the policy-conditioned format the system consumes. A conversion pipeline ingests the Jigsaw Toxic Comment Classification data [17] and the Stormfront-derived HateSpeech18 sentence-level hate-speech corpus [18], and emits records of the form (source, message, rules, context, label). Each message is paired with one of fifteen realistic organizational rule sets spanning respect and anti-harassment, anti-discrimination, anti-spam, on-topic, English-only, and composite multi-rule policies, so

a labeled message is evaluated under a concrete named policy rather than an abstract taxonomy. Human annotation is preserved as ground truth, and no private conversational data is used. The held-out evaluation set contains 300 messages, balanced at 150 allowable and 150 violating.

To isolate what a change in policy content contributes, independent of the model, the same live model evaluates the identical 300 messages under four conditions: a generic safety taxonomy and organizational rules at three sensitivity settings (Strict, Moderate, Relaxed), with no parameter update between conditions. Because all four score the identical items, this is a paired design, which licenses McNemar's test for the significance of any accuracy difference between two conditions in place of an informal comparison of independent accuracies.

Evaluation was run live against the deployed text-moderation plugin, which calls a hosted large open model (OpenAI gpt-oss-120b served via the NVIDIA NIM API) [19] with temperature 0. Greedy decoding at temperature 0 makes the model near-deterministic; for full transparency, no explicit random seed was set in the API call, and no machine-readable run timestamp was recorded, so the run is dated by output file modification time. Reported figures are recomputed directly from the 1,200 stored per-message predictions (300 messages times four conditions). Significance tests use McNemar's paired test, exact binomial when the smaller discordant count is below 25, and the continuity-corrected chi-square otherwise, with 95% confidence intervals from a 10,000-sample bootstrap. The raw predictions and the recomputation scripts are archived with the submission, so every number can be reproduced [20].

This study measures four things directly: the effect of policy content on decisions, the consistency of that mechanism across rule sets, the effect of the sensitivity control, and live policy-evaluation latency; it also reports the standalone coverage of the deterministic pre-filter layer. It does not report end-to-end accuracy on real image and audio media, which requires captioning and transcription of real files and is scoped as future work; the multimodal claim in this paper is architectural, supported by the design and by the literature on text-reduction moderation [6,7,8], and is stated as such.

Results And Discussion

The gatekeeper enforces structurally

A flagged message is excluded from the visibility query that renders the group feed and is never returned to any member other than the sender and administrators. This holds independent of model accuracy: it is a fact about where moderation sits in the send path, verified at the code level, not a statistical outcome that could degrade on a harder input distribution. It is reported first because every result that follows concerns how well the gate decides, not whether the gate holds once it has decided.

Policy content is a controllable variable with a significant effect

Table *2* reports the four metrics on the same 300 messages, scored by the same live model, under a generic safety taxonomy versus the organization's own rules at moderate sensitivity. The effect is concentrated in recall. Reading organizational rules instead of a generic taxonomy raises recall from 46.0% to 76.7%, a gain of 30.7 percentage points, while precision falls more modestly from 83.1% to 76.7%. Net accuracy rises from 68.3% to 76.7%. On the 300 paired predictions, the accuracy difference is significant (McNemar exact $p = 0.004$, 95% bootstrap CI [+3.0, +14.0] percentage points; discordant pairs 48 in favor of the organizational rules against 23 in favor of the generic taxonomy), reported in Table *5*. The practical reading is that a generic safety taxonomy misses more than half of what an organization's own rules would catch: business-language violations, off-topic drift, and English-only breaches, none of which map onto a public-platform harm category. Policy-as-Prompt closes most of that gap with a change of prompt content alone and no parameter update. This is the controlled, single-system version of the pattern established at field scale by the observation that fixed-taxonomy guardrails collapse under free-form policy [4]: here, the model is held constant, and policy content is the only variable that moves. Figure *3* shows the tradeoff and its confidence interval.

Condition	Accuracy	Precision	Recall	F1
Generic taxonomy	68.3%	83.1%	46.0%	59.2%
Organizational rules (moderate)	76.7%	76.7%	76.7%	76.7%
Delta	+8.3 pp	−6.4 pp	+30.7 pp	+17.4 pp

TABLE 2: Moderation performance on the identical 300 held-out messages, same live model, generic safety taxonomy versus organizational rules at moderate sensitivity. Recomputed from stored per-message predictions

pp: percentage points.
Precision: of messages the system flagged, the fraction that were genuine violations.
Recall: of genuine violations, the fraction the system flagged.
F1: the harmonic mean of precision and recall.

All figures are on the same 300 held-out messages, scored by the same model with no parameter change between conditions.

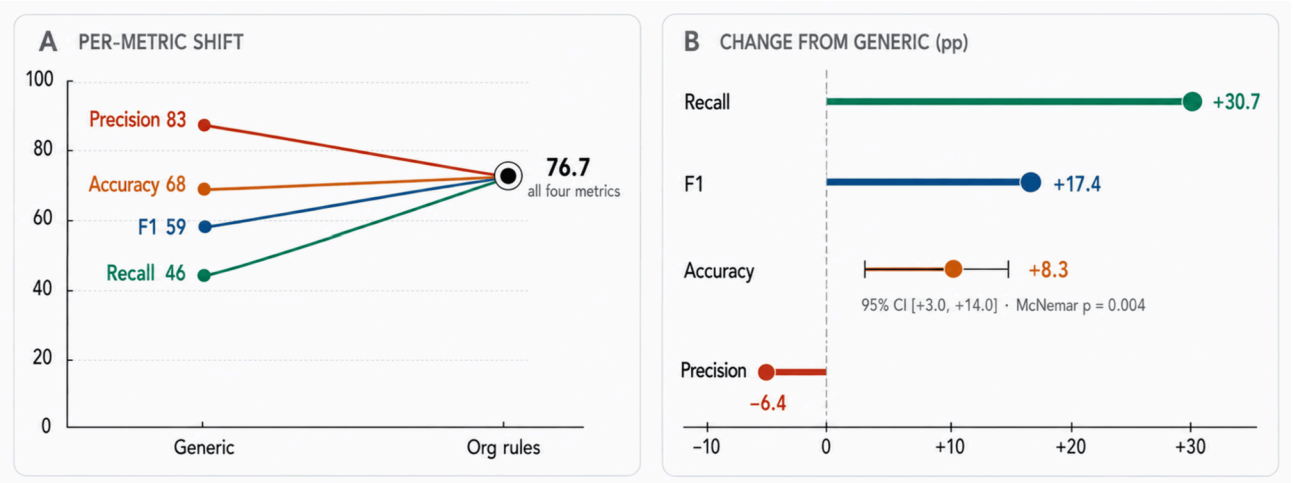


FIGURE 3: Reading organizational rules rebalances the moderator

Policy-content isolation on the same 300 held-out messages, same model, no retraining.
Panel A: each metric shifts from the generic taxonomy to the organizational rules, where all four converge at 76.7%.
Panel B: change from the generic baseline in percentage points; the accuracy interval and significance are shown.
pp: percentage points;
CI: confidence interval.
McNemar's paired test on the 300 matched predictions.

The mechanism generalizes across policies without retraining

The same mechanism was evaluated under all fifteen distinct organizational rule sets in the held-out data, with no per-rule tuning and no model change. Table *3* reports per-rule accuracy, which ranges from 56.5% on the most demanding rule set to 93.8% on the least, with most rule sets between 75% and 94%. The point is architectural rather than a leaderboard: one mechanism, expressed only as different natural-language text in the prompt, enforces fifteen different policies at usable accuracy without a single retraining step. The rule sets where accuracy is lowest, a composite insults-plus-self-promotion-plus-relevance rule at 56.5% and a multi-clause programming-group rule at 57.9%, are the most compositionally complex, which points to prompt decomposition rather than retraining as the natural next improvement, consistent with the mechanism's design.

Rule set (abbreviated)	n	Accuracy	F1 (flagged)
Respectful, no insults	26	92.3%	93.3%
No explicit/violent content	16	93.8%	93.3%
Respect, no harassment/bullying	17	88.2%	87.5%
Community: no hate/threats	20	85.0%	87.0%
English only	15	80.0%	84.2%
Group guidelines: respect, no spam	21	81.0%	77.8%
No spam/unsolicited promotion	17	82.4%	76.9%
No discrimination	19	78.9%	75.0%
Stay on topic	21	76.2%	76.2%
Business: professional language	32	75.0%	75.0%
Study group: on-topic, no aggression	16	75.0%	75.0%
No hate speech/personal attacks	19	68.4%	72.7%
No self-promotion/personal links	19	63.2%	58.8%
Programming group: multi-clause	19	57.9%	55.6%
Composite: insults+promo+relevance	23	56.5%	58.3%

TABLE 3: Cross-policy consistency: per-rule-set accuracy under organizational rules at moderate sensitivity, across all fifteen rule sets, one shared mechanism with no per-rule retraining.

n: number of messages in the held-out set for that rule set.

Enforcement intensity is a second, independent control

Table 4 reports the four metrics across the three sensitivity settings with the organizational rules held fixed. Strict and Moderate are statistically indistinguishable (accuracy 77.0% versus 76.7%; McNemar p = 1.00). Relaxed differs substantially and significantly: accuracy falls to 65.7%, and recall drops from 76.7% to 35.3%, while precision rises to 89.8% (McNemar continuity-corrected p = 0.0008, 95% bootstrap CI [−17.3, −5.0] percentage points against Moderate), in Table 5. The finding is specific. Sensitivity is not a symmetric dial where Strict and Relaxed move equal and opposite amounts from Moderate. It behaves as a one-sided loosening control: tightening from Moderate to Strict buys nothing essentially on this data, while loosening to Relaxed gives up more than a third of the system's recall for a gain in precision. The actionable guidance for an administrator is to treat Moderate as the effective default and reserve Relaxed for channels that deliberately prioritize low-friction, high-precision enforcement over catching every violation. That this control is exposed at all, and that its effect is measurable and interpretable, is the architectural point. Figure 4 shows the sweep and its significance.

Sensitivity	Accuracy	Precision	Recall	F1
Strict	77.0%	76.8%	77.3%	77.1%
Moderate	76.7%	76.7%	76.7%	76.7%
Relaxed	65.7%	89.8%	35.3%	50.7%

TABLE 4: Enforcement intensity sweep: same 300 messages, organizational rules held fixed, varied only by sensitivity setting.

Precision: of messages the system flagged, the fraction that were genuine violations.
Recall: of genuine violations, the fraction the system flagged.
F1: the harmonic mean of precision and recall.

Metrics as defined in Table 2. The organizational rules are held fixed; only the sensitivity setting varies.

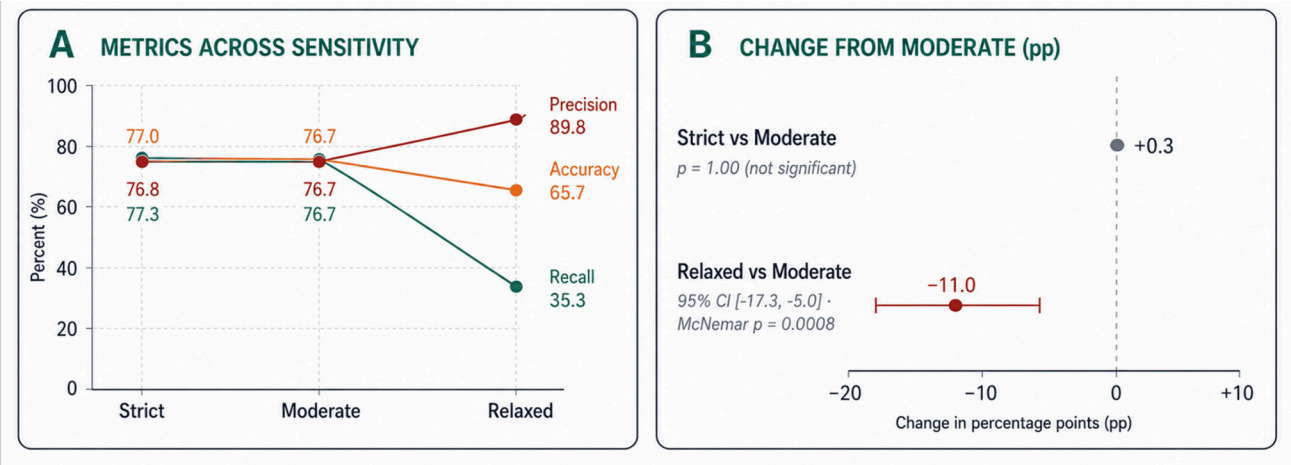


FIGURE 4: Sensitivity sweep (hybrid panel)

Enforcement intensity sweep with organizational rules held fixed.
Panel A: precision, recall, and accuracy across Strict, Moderate, and Relaxed settings.
Panel B: change from Moderate in percentage points, with McNemar significance. Strict versus Moderate is not significant (p = 1.00); Relaxed versus Moderate is significant (p = 0.0008).
pp: percentage points
CI: confidence interval.

Comparison	Δ Accuracy	Discordant (b / c)	p-value	95% CI
Organizational rules vs generic taxonomy	+8.3 pp	48 / 23	0.004 (exact)	[+3.0, +14.0] pp
Strict vs Moderate	+0.3 pp	1 / 0	1.00 (exact)	[+0.0, +1.0] pp
Relaxed vs Moderate	−11.0 pp	29 / 62	0.0008 (χ^2)	[−17.3, −5.0] pp

TABLE 5: Paired McNemar significance tests on the 300 matched predictions per comparison. Exact binomial when the smaller discordant count is below 25; continuity-corrected chi-square otherwise. CIs from a 10,000-sample bootstrap.

pp: percentage points
b / c: the two discordant-pair counts in McNemar's test, where b is the number of messages the first condition classified correctly and the second did not, and c the reverse
χ^2: chi-square test statistic
CI: confidence interval. McNemar's test compares two conditions on the same items

McNemar's test compares two conditions on the same items; a low p-value indicates the accuracy difference is unlikely to be due to chance. The exact binomial test is used when the smaller discordant count is below 25, and the continuity-corrected chi-square otherwise.

The unified text abstraction holds across modalities

A text-bypass check, in which image and audio inputs are reduced to text and then routed through the same text-moderation path, demonstrates a specific and narrower claim than end-to-end multimodal accuracy: that the downstream policy-evaluation step performs consistently once a faithful text reduction exists, regardless of which modality that reduction originated from. It does not demonstrate that the upstream captioning and transcription models themselves produce faithful reductions of real images or real audio; that is a separate, not-yet-measured claim, and the literature on text-reduction as a moderation strategy [6,7,8] is cited as precedent for expecting the full pipeline to recover comparable performance, not as a substitute for measuring it directly.

The plugin architecture runs live in real time

The evaluation was produced by the live-hosted configuration, and the policy-evaluation latency was captured per batch over real API round-trip. Table *6* reports the distribution. Across the four conditions, batched policy evaluation averaged between 436 and 484 ms per ten-message batch, with maxima under 650 ms, against the hosted model. Sub-second batched evaluation at the core of the pipeline demonstrates that the policy evaluator, which every modality funnels into after text reduction, sustains the throughput real-time chat requires. Because latency here is genuine wall-clock timing of live calls rather than an estimate, it is also the concrete evidence behind the plugin architecture's interchangeability claim: this is what the hosted configuration measured like when actually run. Figure *5* shows the per-condition latency against the real-time budget.

Condition	Mean (ms)	Median (ms)	Min (ms)	Max (ms)
Generic taxonomy	436	435	302	584
Organizational (moderate)	470	470	341	591
Organizational (strict)	484	485	349	647
Organizational (relaxed)	455	453	302	648

TABLE 6: Live policy-evaluation latency against the hosted model, per ten-message batch, by condition. Genuine wall-clock round-trip timing from the live benchmark.

Latency is wall-clock time for one round trip to the hosted model per ten-message batch, not per single message. All conditions run on the same held-out set.

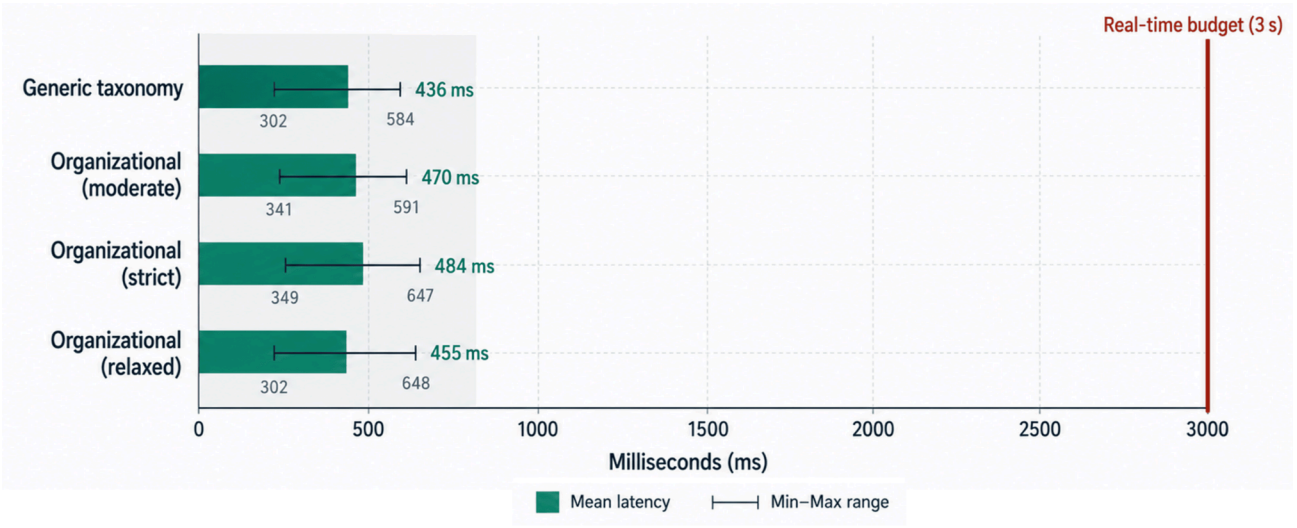


FIGURE 5: Live policy-evaluation latency

Live policy-evaluation latency per ten-message batch, measured as wall-clock time for one round trip to the hosted model. Bars show mean latency; whiskers show the minimum-maximum range. The vertical line marks the three-second real-time budget.
ms: milliseconds
s: seconds

The deterministic and model layers combine to a defensible robustness floor

The deterministic pre-filter is a fast, model-independent first pass, positioned as a supplement to the model layer rather than the primary defense. Measured against a set of adversarial injection prompts and a matched set of benign prompts, the deterministic filter alone detected 40.0% of injection attempts (16 of 40) at a false-positive rate of 20.0% (8 of 40). These figures describe the deterministic layer in isolation. The primary defense against injection is the LLM policy layer behind it, and a full end-to-end evaluation of the two layers combined against a large off-list adversarial corpus is scoped as future work. The architectural contribution is the defense-in-depth arrangement, a zero-cost deterministic screen in front of the model, not a claim that the screen alone is sufficient.

Discussion

The results jointly support one claim: an adaptable, multimodal, pre-delivery moderation system can be assembled from interchangeable off-the-shelf models plus an architecture, with no training in the loop. The gatekeeper's structural enforcement closes the timing gap by construction. The policy-content result shows that the core promise of Policy-as-Prompt, adaptation without retraining, produces a real and significant recall gain rather than a theoretical one. The cross-policy result shows the same mechanism carrying fifteen different policies with no retraining, the property that makes the approach deployable across teams. The sensitivity result shows a second administrator-facing control with a measurable, interpretable, asymmetric effect. The latency result shows the live system operating within real-time bounds and is the empirical content of the plugin architecture's interchangeability claim. Together, these describe a system whose novelty is the arrangement, not any single model.

Table *7* positions Convo-Ease against representative systems across the dimensions that matter for enterprise deployment. Text-only guardrails are strong on speed and configurability but carry no image or audio coverage and no organizational context [1,2]. Multimodal systems extend coverage but pay for it in latency and stop short of audio or of natural-language policy [5,6,9]. None combines real-time text-image-audio coverage with adaptable natural-language policy and pre-delivery enforcement. Commercial cloud moderation services share the same structural limits: predefined non-extensible rules, text-dominant coverage, latency that is unpredictable under chat loads, and a privacy model that

requires sending content to an external server, incompatible with several legitimate enterprise data-handling requirements. Convo-Ease's plugin-based, on-premise-capable design is built directly against that set of constraints, and its policy-adaptability advantage over a fixed taxonomy is a measured result rather than an assertion.

System	Modalities	Real-Time	Policy Adaptability	Org. Focus	Key Innovation	Gap Addressed
Llama Guard [1]	Text	Yes	Zero-shot taxonomy	AI safety	Configurable safety taxonomy	No multimodal, no org context
LlamaFirewall [2]	Text	Yes	Modular policy engine	Agent security	Layered defense	Limited multimodal, no live chat
Policy-as-Prompt [3]	Text	Yes	Natural-language prompts	Community mod.	Rules as model instructions	No multimodal, no live-chat eval
MM-SOC [5]	Text + Image	No	Benchmark only	Social media	Multimodal benchmark suite	No real-time, no custom policy
HateMM [6]	Text+Image+Audio	Limited	Fixed-task	Hate detection	Transcript-based video fusion	No org context, compute-heavy
Llama Guard 3 Vision [9]	Text + Image	Yes	Zero-shot taxonomy	AI safety	Multimodal Llama Guard	No audio, English-only, 1 image
Convo-Ease (this work)	Text+Image+Audio	Yes	NL rules + intensity, no training	Enterprise chat	Gatekeeper + Policy-as-Prompt + plugin backends	Target niche

TABLE 7: Comparative positioning of Convo-Ease against representative moderation systems across enterprise-deployment dimensions.

The modularity supports genuinely different domains through policy edits alone. In education, the system can enforce academic-integrity policies and constrain assignment sharing around deadlines. In corporate settings, it can enforce information-classification policies, professional-communication norms, and jurisdiction-specific compliance. In healthcare, it can govern patient-related discussions and enforce the boundary between non-clinical channels and supervised clinical advice. In each case, the change is to the natural-language policy, not to the model.

Limitations and future work

The limitations are stated plainly. The system inherits the failure modes of its underlying models: occasional hallucination, sensitivity to prompt phrasing, a finite context window, and the inference cost of a large model. The evaluation isolates the policy-content variable well, but is run on a proxy corpus built from public labeled data; real organizational chat has a distribution that the system will meet only in deployment. The multimodal claim is architectural: this study measures the text policy-evaluation core directly but does not report end-to-end accuracy on real image and audio media, and per-modality latency for the captioning and transcription stages is likewise not yet profiled on real files. The deterministic pre-filter's standalone injection coverage is limited by design, and a combined two-layer robustness evaluation against a large off-list adversarial corpus remains to be run. Finally, evaluating against a hosted third-party model exposes a benchmark to that provider's variability, which the system's fail-closed design protects a deployment against, but which complicates a perfectly controlled measurement.

Future work follows three horizons. In the near term, the immediate open items are end-to-end multimodal measurement, real images through the captioner, and real audio through transcription scored against labels, per-modality latency profiling of those stages, and a combined-layer adversarial robustness study. In the medium term, prompt decomposition for compositionally complex policies is the natural accuracy improvement consistent with the no-training design, and video moderation can be added by sampling keyframes through the existing image path with no change to the rest of the pipeline; the plugin architecture also admits optional parameter-efficient fine-tuning [21,22] for organizations that want domain specialization beyond prompt-level policy, which we leave to future work as an extension rather than a requirement. In the long term, deeper explainability with image-region attribution grounded in attribution methods [23,24], user-configurable thresholds alongside organizational policy, longitudinal policy-trend analysis [25], and cross-channel consistency across email, chat, and voice would extend the framework into a uniform organizational moderation layer.

Conclusions

Convo-Ease is an architecture for real-time multimodal moderation in enterprise communication, and a working system that realizes it. Its contribution is the arrangement of four design decisions: a pre-delivery gatekeeper that enforces validation structurally in the send path, a Policy-as-Prompt mechanism that adapts to organizational rules with no training, a plugin-based backend that makes any per-modality model interchangeable through configuration, and a unified text abstraction that lets one policy layer govern text, image, and audio. The central claim is validated under controlled, paired conditions: on the same model and the same 300 messages, reading organizational rules instead of a generic safety taxonomy raises recall by more than thirty percentage points, a significant difference, and the same mechanism carries fifteen distinct policies with no retraining. A second control over enforcement intensity has a measurable, asymmetric, and therefore actionable effect, and the live system evaluates policy within real-time bounds. Convo-Ease shows that the enterprise moderation problem is solvable as a systems problem, by arranging interchangeable off-the-shelf models behind an adaptable policy layer and a structural gate, rather than by training a new model, and that the resulting design occupies a position no existing system fills.

Additional Information

Disclosures

Human subjects: All authors have confirmed that this study did not involve human participants or tissue. **Animal subjects:** All authors have confirmed that this study did not involve animal subjects or tissue. **Conflicts of interest:** In compliance with the ICMJE uniform disclosure form, all authors declare the following: **Payment/services info:** All authors have declared that no financial support was received from any organization for the submitted work. **Financial**

relationships: All authors have declared that they have no financial relationships at present or within the previous three years with any organizations that might have an interest in the submitted work. **Other relationships:** All authors have declared that there are no other relationships or activities that could appear to have influenced the submitted work.

Data Availability Statements

The data (and/or code) supporting this study are openly available in Zenodo (Convo-Ease v1.0.0) at 10.5281/zenodo.21128761, reference 21128761.

References

1. Inan H, Upasani K, Chi J, et al.: Llama Guard: LLM-based input-output safeguard for human-AI conversations [PREPRINT]. arXiv.. 2023, 10.48550/arXiv.2312.06674
2. Chennabasappa S, Nikolaidis C, Song D, et al.: LlamaFirewall: an open source guardrail system for building secure AI agents [PREPRINT]. arXiv.. 2025, 10.48550/arXiv.2505.03574
3. Palla K, García JLR, Hervás A, Khan H, Watson D: Policy-as-prompt: rethinking content moderation in the age of large language models [PREPRINT]. arXiv.. 2025, 10.48550/arXiv.2502.18695
4. Hoover M, Baherwani V, Jain N, et al.: DynaGuard: a dynamic guardian model with user-defined policies [PREPRINT]. arXiv.. 2025, 10.48550/arXiv.2509.02563
5. Jin Y, Choi M, Verma G, Wang J, Kumar S: MM-SOC: benchmarking multimodal large language models in social media platforms. Findings of the Association for Computational Linguistics: ACL 2024.. 2024, 10.18653/v1/2024.findings-acl.367
6. Koushik GA, Kanojia D, Treharne H: Towards a robust framework for multimodal hate detection: a study on video vs image-based content. Companion Proceedings of the ACM Web Conference 2025.. 2025, 10.1145/3701716.3715878
7. Kearns L: Content moderation assistance through image caption generation. Intell Syst Appl.. 2024, 21:10.1016/j.iswa.2024.200350
8. Gou Y, Chen K, Liu Z, et al.: Eyes closed, safety on: protecting multimodal LLMs via image-to-text transformation. Computer Vision - ECCV 2024.. 2024, 15075:388-404. 10.1007/978-3-031-72643-9_23
9. Chi J, Karn U, Zhan H, et al.: Llama Guard 3 Vision: safeguarding human-AI image understanding conversations [PREPRINT]. arXiv.. 2024, 10.48550/arXiv.2411.10414
10. Luo W, Ma S, Liu X, Guo X, Xiao C: JailBreakV-28K: a benchmark for assessing the robustness of multimodal large language models against jailbreak attacks [PREPRINT]. arXiv.. 2024, 10.48550/arXiv.2404.03027
11. Kang M, Chen Z, Xu C, et al.: GuardSet-X: massive multi-domain safety policy-grounded guardrail dataset [PREPRINT]. arXiv.. 2025, 10.48550/arXiv.2506.19054
12. Radford A, Kim JW, Xu T, et al.: Robust speech recognition via large-scale weak supervision. Proceedings of the 40th International Conference on Machine Learning.. 2023, 202:28492-518. 10.48550/arXiv.2212.04356
13. Chen S, Wu Y, Wang C, et al.: Evaluating OpenAI's Whisper ASR: performance analysis across diverse speech recognition tasks. JASA Express Lett.. 2024, 4:025204. 10.1121/10.0025467
14. Li J, Li D, Savarese S, Hoi S: BLIP-2: bootstrapping language-image pre-training with frozen image encoders and large language models. Proceedings of the 40th International Conference on Machine Learning.. 2023, 202:19730-42. 10.48550/arXiv.2301.12597
15. Guo W, Wang J, Wang S: Deep multimodal representation learning: a survey. IEEE Access.. 2019, 7:63373-94. 10.1109/ACCESS.2019.2916887
16. Brown TB, Mann B, Ryder N, et al.: Language models are few-shot learners [PREPRINT]. Advances in Neural Information Processing Systems 33.. 2020, 1877-901. 10.48550/arXiv.2005.14165
17. Jigsaw, Conversation AI: Toxic Comment Classification Challenge. 2018, Accessed: November 10, 2025: https://www.kaggle.com/competitions/jigsaw-toxic-comment-classification-challenge.

18. de Gibert O, Perez N, García-Pablos A, Cuadros M: Hate speech dataset from a white supremacy forum. Proceedings of the 2nd Workshop on Abusive Language Online (ALW2).. 2018, 11-20. 10.18653/v1/W18-5102

19. Nvidia NIM: gpt-oss-120b. (2026). Accessed: February 18, 2026: https://build.nvidia.com/openai/gpt-oss-120b.

20. Jadhav SS: Convo-Ease (v1.0.0-paper). 2026, 10.5281/zenodo.21128761

21. Hu EJ, Shen Y, Wallis P, et al.: LoRA: low-rank adaptation of large language models [PREPRINT]. International Conference on Learning Representations. 2022, 10.48550/arXiv.2106.09685

22. Dettmers T, Pagnoni A, Holtzman A, Zettlemoyer L: QLoRA: efficient finetuning of quantized LLMs. Advances in Neural Information Processing Systems 36.. 2023, 10.48550/arXiv.2305.14314

23. Lundberg SM, Lee S-I: A unified approach to interpreting model predictions. Advances in Neural Information Processing Systems 30.. 2017, 4765-74. 10.48550/arXiv.1705.07874

24. Rodis N, Sardianos C, Papadopoulos GT, et al.: Multimodal explainable artificial intelligence: a comprehensive review of methodological advances and future research directions. IEEE Access.. 2024, 12:159794-820. 10.1109/ACCESS.2024.3386886

25. Nedungadi P, Tang KY, Menon RRK, Raman R: AI techniques and applications for online social networks and media: insights from BERTopic modeling. IEEE Access.. 2025, 13:23456-78. 10.1109/ACCESS.2025.3354432